CSE 2015- Data Analysis and Visualization

Module 1-Introduction to Data Analysis

Module 1: Introduction to Data Visualization [12 Hrs] [Bloom's Level Selected: Understand]

Data collection, Data Preparation Basic Models- Overview of data visualization - Data Abstraction - Task Abstraction - Analysis: Four Levels for Validation, Interacting with Databases, Data Cleaning and Preparation, Handling Missing Data, Data Transformation.

Python Libraries: NumPy, pandas, matplotlib, GGplot, Introduction to pandas Data Structures .

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
40 YEARS

Introducing Data

- Facts and statistics collected together for reference or analysis
- Data has to be transformed into a form that is efficient for movement or processing.

Over view of Data Analysis

- **Data analysis** is defined as a process of cleaning, transforming, and modeling data to discover useful information for business decision-making.
- The purpose of Data Analysis is to extract useful information from data and taking the decision based upon the data analysis.
- A simple example of Data analysis is whenever we take any decision in our day-to-day life is by thinking about what happened last time or what will happen by choosing that particular decision.

- This is nothing but analyzing our past or future and making decisions based on it.
- For that, we gather memories of our past or dreams of our future.
- So that is nothing but data analysis. Now same thing analyst does for business purposes, is called Data Analysis.

Data Analysis Tools

Data in the Real World

Clinical
Demographics, EHR Data, Lab Test Results, Diagnoses, Procedures, Pathology/ Histology Data, Radiology Images, Microbiology Data, Provider Notes, Admission/ Discharge and Progress Reports, Performance Status

Medication
Medication Orders, Administration (Dose, Route, NDC/RxNorm codes), Concomitant Therapies, Point of Sale Data, (Prescription & OTC) Prescription Refill, Allergies

Claims
Medical Claims, Prescription Drug Claims, Other Drug and Treatment Use Data

Molecular Profiling
Genomic and Genetic Testing Data (SNPs/Panels), Multi-Omics Data (Proteomics, Transcriptomics , Metabonomics, Lipidomics), Other Biomarker Status

Family History
Historical Data on Health Conditions and Allergies Relating to Patient and Extended Family, Smoking Status, Alcohol Use

Mobile Health
Fitness Trackers, Wearable Devices, Other Health Apps Measuring Activity and Body Function

Environmental
Climate Factors, Pollutants, Infections, Lifestyle Factors (diets, stress), Other Environmental and Occupational Sources

Patient Reported
Patient Reported Outcomes, Surveys, Diaries (diets, habits), Personal Health Records, Adverse Event Reporting, Quality of Life Measures

Social Media
Patient Communities, Twitter, Facebook, Blogs

Literature
Disease Burden, Clinical Characteristics, Prevalence/Incidence, Rates of Treatment, Resource Use and Costs, Disease Control, Quality of Life Measures

DATA VS. INFORMATION

Examples of Data

- 120/80 blood pressure reading
- Date on an employee application
- Number of cesarean sections in May
- Vendor address

Examples of Information

- John Doe's blood pressure reading on 9/15/15
- Employee application record
- ABC Hospital cesarean section rate for May
- Vendor record

Example Data Governance Functions

- » Data Quality Control and Management
- » Identity Management
- » Data Cleansing
- » Metadata Management
- » Master Data Management

Example Information Governance Functions

- » Enterprise-wide Governance Policies
- » Life Cycle Management
- » Information Use, Exchange, and Preservation
- » Physical and Electronic Systems Governance
- » Privacy and Security
- » Information Risk Management
- » Legal and Regulatory Response
- » Information Vendor Services Management

Data Collection

Data collection is the process of gathering and collecting information from various sources to analyse and make informed decisions based on the data collected. This can involve various methods, such as surveys, interviews, experiments, and observation.

Types of Data Collection

- Primary Data Collection
  - Primary data collection is the process of gathering original and firsthand information directly from the source or target population.
- Secondary Data Collection
  - Secondary data collection is the process of gathering information from existing sources that have already been collected and analyzed by someone else, rather than conducting new research to collect primary data.
- Qualitative Data Collection
  - Qualitative data collection is used to gather non-numerical data such as opinions, experiences, perceptions, and feelings, through techniques such as interviews, focus groups, observations, and document analysis.
- Quantitative Data Collection
  - Quantitative data collection is a used to gather numerical data that can be analyzed using statistical methods. This data is typically collected through surveys, experiments, and other structured data collection methods.

Data Collection Methods

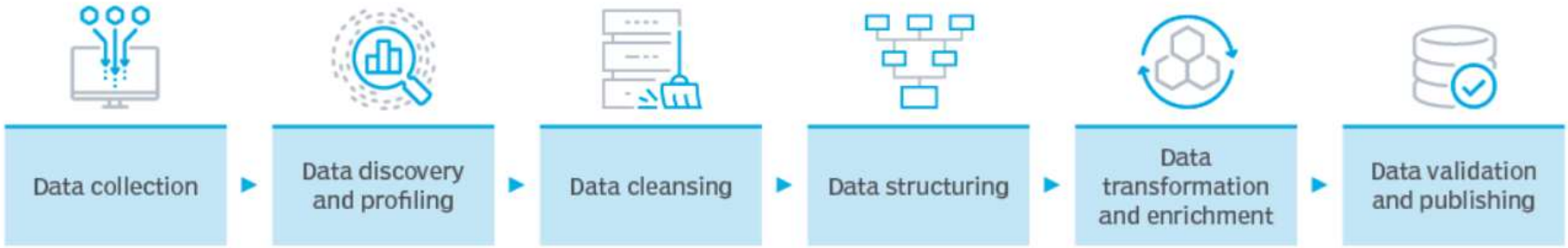
- Surveys
  - Surveys involve asking questions to a sample of individuals or organizations to collect data. Surveys can be conducted in person, over the phone, or online.
- Interviews
  - Interviews involve a one-on-one conversation between the interviewer and the respondent. Interviews can be structured or unstructured and can be conducted in person or over the phone.
- Focus Groups
  - Focus groups are group discussions that are moderated by a facilitator. Focus groups are used to collect qualitative data on a specific topic.

- Observation
  - Observation involves watching and recording the behavior of people, objects, or events in their natural setting. Observation can be done overtly or covertly, depending on the research question.
- Experiments
  - Experiments involve manipulating one or more variables and observing the effect on another variable. Experiments are commonly used in scientific research.
- Case Studies
  - Case studies involve in-depth analysis of a single individual, organization, or event. Case studies are used to gain detailed information about a specific phenomenon.

Data Preparation

- Data preparation is the process of gathering, combining, structuring and organizing data so it can be used in business intelligence (BI), analytics and data visualization applications.

Key data preparation steps



PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
40 YEARS

- **Data Collection-** Relevant data is gathered from operational systems, data warehouses, data lakes and other data sources.
- **Data Discovery and Profiling-** The next step is to explore the collected data to better understand what it contains and what needs to be done to prepare it for the intended uses.
- **Data Cleansing-** Next, the identified data errors and issues are corrected to create complete and accurate data sets.
- **Data Structuring-** At this point, the data needs to be modeled and organized to meet the analytics requirements.

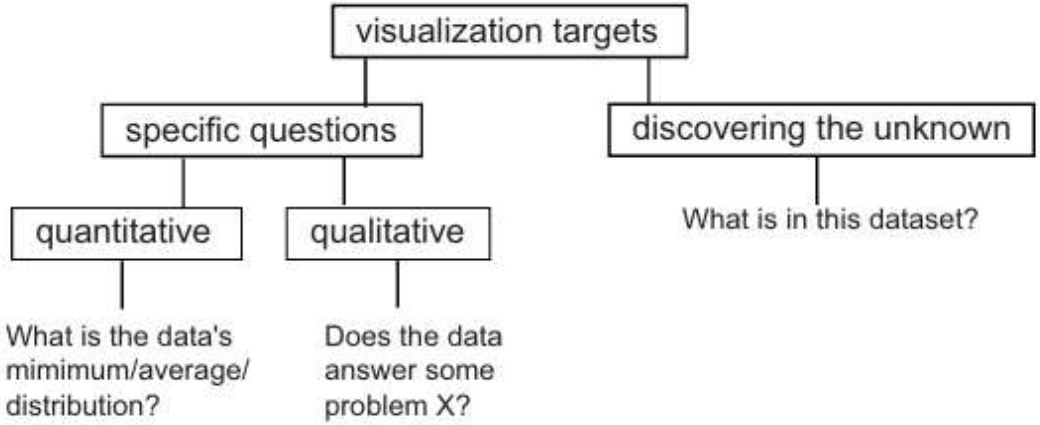
- **Data Transformation and Enrichment.**Data enrichment further enhances and optimizes data sets as needed, through measures such as augmenting and adding data.
- **Data Validation and Publishing.** In this last step, automated routines are run against the data to validate its consistency, completeness and accuracy. The prepared data is then stored in a data warehouse, a data lake or another repository

Benefits of Data Preparation

- Ensure the data used in analytics applications produces reliable results
- Identify and fix data issues that otherwise might not be detected
- Enable more informed decision-making by business executives and operational workers
- Reduce data management and analytics costs
- Avoid duplication of effort in preparing data for use in multiple applications
- Get a higher ROI from BI and analytics initiatives.

Overview of Data Visualization

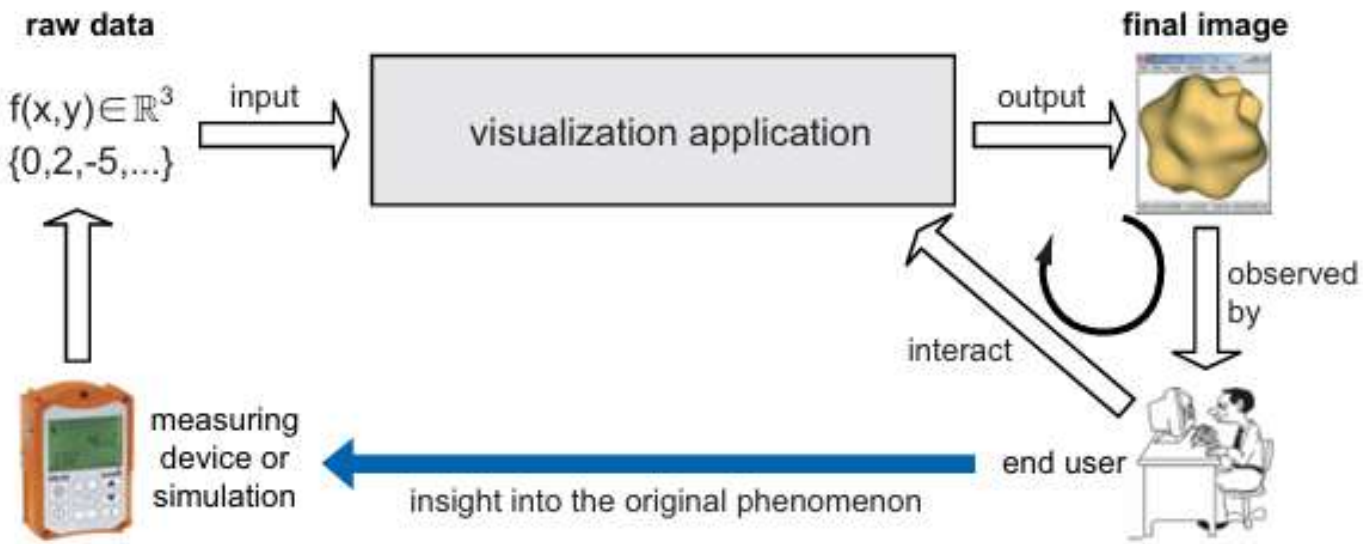
- The purpose of visualization is to get insight, by means of interactive graphics, into various aspects related to some process we are interested in, such as a scientific simulation or some real-world process.



Questions Targeted by the Visualization process

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
40 YEARS

Conceptual View of Visualization Process



PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
40 YEARS

Data Abstraction

- Data abstraction is the process of concealing irrelevant or unwanted data from the end user. Data Abstraction is a concept that allows us to store and manipulate data in a more abstract, efficient way. This type of abstraction separates the logical representation of data from its physical storage, giving us the ability to focus solely on the important aspects of data without being bogged down by the details.

Challenges with Data Abstraction

Understanding Data Complexity

Data abstraction requires an understanding of both complex data structures and logical rules. Although abstracting data can involve simplifying it for easier management purposes, this doesn't necessarily mean less complexity.

Hiding Details while Remaining Accurate

Data abstraction is also a way to hide certain details from view without compromising accuracy or security.

Limitations of Schemas and Abstraction Layers

When it comes to documenting large datasets, predefined schemas are often used as an easy way to structuralize the data correctly.

Benefits of Data Abstraction

- **Efficiency**: Abstraction allows us to manipulate data in a more abstract way, separating logical representation from physical storage.
- **Focus on Essentials**: By ignoring unnecessary details, we can concentrate on what truly matters.
- **System Efficiency**: Users access relevant data without hassle, and the system operates efficiently

What is Data Validation?

Data validation refers to the process of ensuring the accuracy and quality of data. It is implemented by building several checks into a system or report to ensure the logical consistency of input and stored data.

Types of Data Validation

1. Data Type Check

- A data type check confirms that the data entered has the correct data type. For example, a field might only accept numeric data. If this is the case, then any data containing other characters such as letters or special symbols should be rejected by the system.

2. Code Check

- A code check ensures that a field is selected from a valid list of values or follows certain formatting rules. For example, it is easier to verify that a postal code is valid by checking it against a list of valid codes. The same concept can be applied to other items such as country codes and NAICS industry codes.

3. Range Check

A range check will verify whether input data falls within a predefined range. For example, latitude and longitude are commonly used in geographic data. A latitude value should be between -90 and 90, while a longitude value must be between -180 and 180. Any values out of this range are invalid.

4. Format Check

Many data types follow a certain predefined format. A common use case is date columns that are stored in a fixed format like “YYYY-MM-DD” or “DD-MM-YYYY.” A data validation procedure that ensures dates are in the proper format helps maintain consistency across data and through time.

5. Consistency Check

- A consistency check is a type of logical check that confirms the data's been entered in a logically consistent way. An example is checking if the delivery date is after the shipping date for a parcel.

6. Uniqueness Check

- Some data like IDs or e-mail addresses are unique by nature. A database should likely have unique entries on these fields. A uniqueness check ensures that an item is not entered multiple times into a database.

What is Data Cleaning?

- Data cleaning is the process of fixing or removing incorrect, corrupted, incorrectly formatted, duplicate, or incomplete data within a dataset. When combining multiple data sources, there are many opportunities for data to be duplicated or mislabeled.

- If data is incorrect, outcomes and algorithms are unreliable, even though they may look correct.

What is the difference between Data Cleaning and Data Transformation?

- Data cleaning is the process that removes data that does not belong in your dataset. Data transformation is the process of converting data from one format or structure into another.
- Transformation processes can also be referred to as data wrangling, or data munging, transforming and mapping data from one "raw" data form into another format for warehousing and analyzing.

Data Cleaning Steps

Step 1: Remove duplicate or irrelevant observations

Step 2: Fix structural errors

Structural errors are when you measure or transfer data and notice strange naming conventions, typos, or incorrect capitalization. These inconsistencies can cause mislabelled categories or classes.

Step 3: Filter unwanted outliers

Step 4: Handle missing data

You can't ignore missing data because many algorithms will not accept missing values.

Step 5: Validate and QA

At the end of the data cleaning process, you should be able to answer these questions as a part of basic validation:

- Does the data make sense?
- Does the data follow the appropriate rules for its field?
- Does it prove or disprove your working theory, or bring any insight to light?
- Can you find trends in the data to help you form your next theory?
- If not, is that because of a data quality issue?

Data Transformation

1. Removing Duplicates Duplicate rows may be found in a DataFrame for any number of reasons. Here is an example:

```
In [45]: data = pd.DataFrame({'k1': ['one', 'two'] * 3 + ['two'],
   ....:                      'k2': [1, 1, 2, 3, 3, 4, 4]})

In [46]: data
Out[46]:
    k1  k2
0  one   1
1  two   1
2  one   2
3  two   3
4  one   3
5  two   4
6  two   4
```

The DataFrame method `duplicated` returns a boolean Series indicating whether each row is a duplicate (has been observed in a previous row) or not:

```
In [47]: data.duplicated()
Out[47]:
0    False
1    False
2    False
3    False
4    False
5    False
6     True
dtype: bool
```

Relatedly, `drop_duplicates` returns a DataFrame where the `duplicated` array is `False`:

Relatedly, drop_duplicates returns a DataFrame where the duplicated array is False:

```
In [48]: data.drop_duplicates()
Out[48]:
    k1  k2
0  one   1
1  two   1
2  one   2
3  two   3
4  one   3
5  two   4
```

Suppose we had an additional column of values and wanted to filter duplicates only based on the 'k1' column:

```
In [49]: data['v1'] = range(7)

In [50]: data.drop_duplicates(['k1'])
Out[50]:
    k1  k2  v1
0  one   1   0
1  two   1   1
```

2. Transforming Data Using a Function or Mapping

- Consider the following hypothetical data collected about various kinds of meat:

```
In [52]: data = pd.DataFrame({'food': ['bacon', 'pulled pork', 'bacon',
   ....:                               'Pastrami', 'corned beef', 'Bacon',
   ....:                               'pastrami', 'honey ham', 'nova lox'],
   ....:                      'ounces': [4, 3, 12, 6, 7.5, 8, 3, 5, 6]})

In [53]: data
Out[53]:
          food  ounces
0        bacon     4.0
1  pulled pork     3.0
2        bacon    12.0

3     Pastrami     6.0
4  corned beef     7.5
5        Bacon     8.0
6     pastrami     3.0
7    honey ham     5.0
8     nova lox     6.0
```

- Suppose you wanted to add a column indicating the type of animal that each food came from. Let's write down a mapping of each distinct meat type to the kind of animal:

```
meat_to_animal = {
  'bacon': 'pig',
  'pulled pork': 'pig',
  'pastrami': 'cow',
  'corned beef': 'cow',
  'honey ham': 'pig',
  'nova lox': 'salmon'
}
```

```
In [55]: lowercased = data['food'].str.lower()

In [56]: lowercased
Out[56]:
0          bacon
1    pulled pork
2          bacon
3       pastrami
4    corned beef
5          bacon
6       pastrami
7      honey ham
8       nova lox
Name: food, dtype: object
```

The map method on a Series accepts a function or dict-like object containing a mapping. We need to convert each value to lowercase using the str.lower Series method:

```
In [57]: data['animal'] = lowercased.map(meat_to_animal)

In [58]: data
Out[58]:
          food  ounces  animal
0        bacon     4.0     pig
1  pulled pork     3.0     pig
2        bacon    12.0     pig
3     Pastrami     6.0     cow
4  corned beef     7.5     cow
5        Bacon     8.0     pig
6     pastrami     3.0     cow
```

3. Replacing Multiple Values

If you want to replace multiple values at once, you instead pass a list and then the substitute value:

```
In [63]: data.replace([-999, -1000], np.nan)
Out[63]:
0    1.0
1    NaN
2    2.0
3    NaN
4    NaN
5    3.0
dtype: float64
```

To use a different replacement for each value, pass a list of substitutes:

```
In [64]: data.replace([-999, -1000], [np.nan, 0])
Out[64]:
0    1.0
1    NaN
2    2.0
3    NaN
4    0.0
5    3.0
dtype: float64
```

The argument passed can also be a dict:

```
In [65]: data.replace({-999: np.nan, -1000: 0})
Out[65]:
0    1.0
1    NaN
2    2.0
3    NaN
4    0.0
5    3.0
dtype: float64
```

Copyright ©2011
Pearson Education

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013

4. Renaming Axis Indexes

Like values in a Series, axis labels can be similarly transformed by a function or mapping of some form to produce new, differently labeled objects.

```
In [66]: data = pd.DataFrame(np.arange(12).reshape((3, 4)),
   ....:                     index=['Ohio', 'Colorado', 'New York'],
   ....:                     columns=['one', 'two', 'three', 'four'])
```

Like a Series, the axis indexes have a `map` method:

```
In [67]: transform = lambda x: x[:4].upper()

In [68]: data.index.map(transform)
Out[68]: Index(['OHIO', 'COLO', 'NEW '], dtype='object')
```

You can assign to `index`, modifying the DataFrame in-place:

```
In [69]: data.index = data.index.map(transform)

In [70]: data
Out[70]:
      one  two  three  four
OHIO    0    1      2     3
COLO    4    5      6     7
NEW     8    9     10    11
```

5. Discretization and Binning

Continuous data is often discretized or otherwise separated into "bins" for analysis. Suppose you have data about a group of people in a study, and you want to group them into discrete age buckets

ages = [20, 22, 25, 27, 21, 23, 37, 31, 61, 45, 41, 32]

Let's divide these into bins of 18 to 25, 26 to 35, 36 to 60, and finally 61 and older. To do so, you have to use cut, a function in pandas:

```
In [76]: bins = [18, 25, 35, 60, 100]

In [77]: cats = pd.cut(ages, bins)

In [78]: cats
Out[78]:
[(18, 25], (18, 25], (18, 25], (25, 35], (18, 25], ..., (25, 35], (60, 100], (35,
 60], (35, 60], (25, 35]]
Length: 12
Categories (4, interval[int64]): [(18, 25] < (25, 35] < (35, 60] < (60, 100]]
```

```
In [79]: cats.codes
Out[79]: array([0, 0, 0, 1, 0, 0, 2, 1, 3, 2, 2, 1], dtype=int8)

In [80]: cats.categories
Out[80]:
IntervalIndex([(18, 25], (25, 35], (35, 60], (60, 100]]
              closed='right',
              dtype='interval[int64]')

In [81]: pd.value_counts(cats)
Out[81]:
(18, 25]     5
(35, 60]     3
(25, 35]     3
(60, 100]    1
dtype: int64
```

6. Detecting and Filtering Outliers

Filtering or transforming outliers is largely a matter of applying array operations. Consider a DataFrame with some normally distributed data:

```
In [92]: data = pd.DataFrame(np.random.randn(1000, 4))

In [93]: data.describe()
Out[93]:
                0            1            2            3
count  1000.000000  1000.000000  1000.000000  1000.000000
mean      0.049091     0.026112    -0.002544    -0.051827
std       0.996947     1.007458     0.995232     0.998311
min      -3.645860    -3.184377    -3.745356    -3.428254
25%      -0.599807    -0.612162    -0.687373    -0.747478
50%       0.047101    -0.013609    -0.022158    -0.088274
75%       0.756646     0.695298     0.699046     0.623331
max       2.653656     3.525865     2.735527     3.366626
```

Suppose you wanted to find values in one of the columns exceeding 3 in absolute value:

```
In [94]: col = data[2]

In [95]: col[np.abs(col) > 3]
Out[95]:
41    -3.399312
136   -3.745356
Name: 2, dtype: float64
```

To select all rows having a value exceeding 3 or –3, you can use the any method on a boolean DataFrame:

```
In [96]: data[(np.abs(data) > 3).any(1)]
Out[96]:
            0         1         2         3
41   0.457246 -0.025907 -3.399312 -0.974657
60   1.951312  3.260383  0.963301  1.201206
136  0.508391 -0.196713 -3.745356 -1.520113
235 -0.242459 -3.056990  1.918403 -0.578828
258  0.682841  0.326045  0.425384 -3.428254
322  1.179227 -3.184377  1.369891 -1.074833
544 -3.548824  1.553205 -2.186301  1.277104
635 -0.578093  0.193299  1.397822  3.366626
782 -0.207434  3.525865  0.283070  0.544635
803 -3.645860  0.255475 -0.549574 -1.907459
```

7.Computing Indicator/Dummy Variables

Another type of transformation for statistical modeling or machine learning applications is converting a categorical variable into a "dummy" or "indicator" matrix.

```
In [109]: df = pd.DataFrame({'key': ['b', 'b', 'a', 'c', 'a', 'b'],
   .....:                    'data1': range(6)})

In [110]: pd.get_dummies(df['key'])
Out[110]:
   a  b  c
0  0  1  0
1  0  1  0
2  1  0  0
3  0  0  1
4  1  0  0
5  0  1  0
```

NumPy

NumPy is a Python package. It stands for 'Numerical Python'. It is a library consisting of multidimensional array objects and a collection of routines for processing of array.

Numeric, the ancestor of NumPy, was developed by Jim Hugunin. Another package Numarray was also developed, having some additional functionalities. In 2005, Travis Oliphant created NumPy package by incorporating the features of Numarray into Numeric package.

Operations using NumPy

Using NumPy, a developer can perform the following operations –

- Mathematical and logical operations on **arrays**.
- Fourier transforms and routines for shape manipulation.
- Operations related to linear algebra.
- NumPy has in-built functions for linear algebra and random number generation.

NumPy – A Replacement for MatLab

NumPy is often used along with packages like **SciPy** (Scientific Python) and **Mat–plotlib** (plotting library).

This combination is widely used as a replacement for MatLab, a popular platform for technical computing.

However, Python alternative to MatLab is now seen as a more modern and complete programming language.

It is open source, which is an added advantage of NumPy.

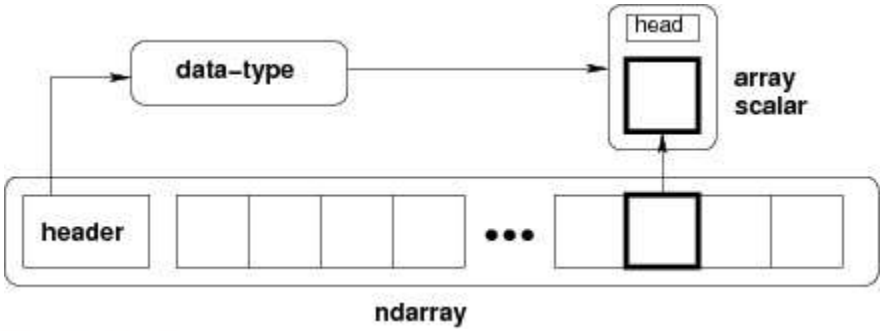
NumPy package is imported using the following syntax −

```
import numpy as np
```

The most important object defined in NumPy is an N-dimensional array type called **ndarray**. It describes the collection of items of the same type. Items in the collection can be accessed using a zero-based index.

Every item in an ndarray takes the same size of block in the memory. Each element in ndarray is an object of data-type object (called **dtype**).

The following diagram shows a relationship between ndarray, data type object (dtype) and array scalar type −



```
numpy.array(object, dtype = None, copy = True, order = None, subok =
False, ndmin = 0)
```

The above constructor takes the following parameters −

Sr.No.	Parameter & Description
1	**object** Any object exposing the array interface method returns an array, or any (nested) sequence.
2	**dtype** Desired data type of array, optional
3	**copy** Optional. By default (true), the object is copied
4	**order** C (row major) or F (column major) or A (any) (default)
5	**subok** By default, returned array forced to be a base class array. If true, sub-classes passed through
6	**ndmin** Specifies minimum dimensions of resultant array

Example:

```
import numpy as np
a = np.array([1,2,3])
print a
```

```
# more than one dimensions
import numpy as np
a = np.array([[1, 2], [3, 4]])
print a
```

Example:

```
# minimum dimensions
import numpy as np
a = np.array([1, 2, 3,4,5], ndmin = 2)
print a
```

```
# dtype parameter
import numpy as np
a = np.array([1, 2, 3],
dtype = complex)
print a
```

NumPy - Array Attributes

ndarray.shape:

This array attribute returns a tuple consisting of array dimensions. It can also be used to resize the array.

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
print a.shape
```

```
# this resizes the ndarray
mport numpy as np
a = np.array([[1,2,3],[4,5,6]])
a.shape = (3,2)
print a
```

NumPy - Array Attributes

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
b = a.reshape(3,2)
print b
```

ndarray.ndim:

This array attribute returns the number of array dimensions.

```
# an array of evenly spaced numbers
import numpy as np
a = np.arange(24)
print a
```

NumPy - Array Attributes

```
# this is one dimensional array
import numpy as np
a = np.arange(24) a.ndim

# now reshape it
b = a.reshape(2,4,3)
print b
# b is having three dimensions
```

numpy.itemsize:

This array attribute returns the length of each element of array in bytes.

NumPy - Array Attributes

numpy.itemsize:
This array attribute returns the length of each element of array in bytes.

```
# dtype of array is int8 (1 byte)
import numpy as np
x = np.array([1,2,3,4,5], dtype = np.int8)
print x.itemsize
```

```
# dtype of array is now float32 (4 bytes)
import numpy as np
x = np.array([1,2,3,4,5], dtype = np.float32)
print x.itemsize
```

NumPy - Array Creation Routines

A new **ndarray** object can be constructed by any of the following array creation routines or using a low-level ndarray constructor.

numpy.empty:

It creates an uninitialized array of specified shape and dtype. It uses the following constructor –

```
numpy.empty(shape, dtype = float, order = 'C')
```

Sr.No.	Parameter & Description
1	**Shape:** Shape of an empty array in int or tuple of int
2	**Dtype:** Desired output data type. Optional
3	**Order:** 'C' for C-style row-major array, 'F' for FORTRAN style column-major array

NumPy - Array Creation Routines

A new **ndarray** object can be constructed by any of the following array creation routines or using a low-level ndarray constructor.

numpy.empty:
It creates an uninitialized array of specified shape and dtype. It uses the following constructor –

```
numpy.empty(shape, dtype = float, order = 'C')
```

Sr.No.	Parameter & Description
1	**Shape:** Shape of an empty array in int or tuple of int
2	**Dtype:** Desired output data type. Optional
3	**Order:** 'C' for C-style row-major array, 'F' for FORTRAN style column-major array

NumPy - Array Creation Routines

```
import numpy as np
x = np.empty([3,2], dtype = int)
print x
```

numpy.zeros:
Returns a new array of specified size, filled with zeros.

```
numpy.zeros(shape, dtype = float, order = 'C')
```

Sr.No.	Parameter & Description
1	**Shape:** Shape of an empty array in int or sequence of int
2	**Dtype:** Desired output data type. Optional
3	**Order:** 'C' for C-style row-major array, 'F' for FORTRAN style column-major array

NumPy - Array Creation Routines

```
# array of five zeros. Default dtype is float
import numpy as np
x = np.zeros(5)
print x
```

```
import numpy as np
x = np.zeros((5,), dtype = np.int)
print x
```

```
# custom type
import numpy as np x = np.zeros((2,2), dtype = [('x', 'i4'), ('y', 'i4')])
print x
```

numpy.ones:

Returns a new array of specified size and type, filled with ones.

```
numpy.ones(shape, dtype = None, order = 'C')
```

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
40 YEARS

NumPy - Array From Existing Data

numpy.as array:
This function is similar to numpy.array except for the fact that it has fewer parameters. This routine is useful for converting Python sequence into ndarray.

numpy.asarray(a, dtype = None, order = None)

Sr.No.	Parameter & Description
1	**a** Input data in any form such as list, list of tuples, tuples, tuple of tuples or tuple of lists
2	**dtype** By default, the data type of input data is applied to the resultant ndarray
3	**order** C (row major) or F (column major). C is default

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
40 YEARS

pandas

Data Cleaning

Pandas - Cleaning Data

```
Fixing bad data in data set. It could be:
-Empty cells
-Data in wrong format
-Wrong data
-Duplicates
```

Cleaning Empty Cells

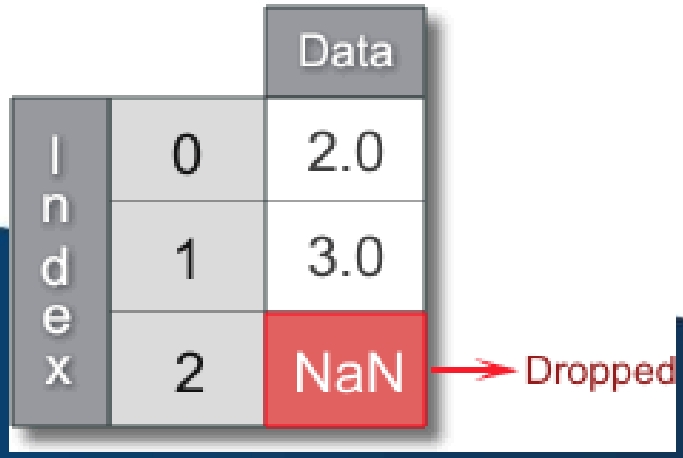
Remove Rows

```python
import pandas as pd
```

```python
df = pd.read_csv('data.CSV')
```

```python
df
```

dropna( )



PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
40 YEARS

	Product Name	Sale Price	Mrp	Number Of Ratings	Sale Date
0	APPLE iPhone 8 Plus (Gold, 64 GB)	49900.0	49900.0	3431	'14/1/2023'
1	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	1.0	3431	'15/1/2023'
2	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	84900.0	3431	'16/1/2023'
3	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	11202	'17/1/2023'
4	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	11202	'17/1/2023'
5	APPLE iPhone 8 Plus (Silver, 64 GB)	NaN	49900.0	3431	'19/1/2023'
6	APPLE iPhone 8 Plus (Space Grey, 64 GB)	49900.0	NaN	3431	20/1/2023
7	APPLE iPhone 8 (Space Grey, 256 GB)	77000.0	77000.0	11202	NaN
8	APPLE iPhone XS Max (Silver, 64 GB)	89900.0	89900.0	1454	'22/1/2023'

dropna()

```
remove rows that contain empty cells / null values
```

```python
#row 5,6,7 deleted
df.dropna()
```

data.dropna()

One	Two
0	2
1	3
2	0
NaN	1

→

One	Two
0	2
1	3
2	0

data.dropna(axis=1)

One	Two
0	2
1	3
2	0
NaN	1

→

Two
2
3
0
1

Drop missing values

PRESIDENCY UNIVERSITY
GAIN MORE KNOWLEDGE REACH GREATER HEIGHTS
Private University Estd. in Karnataka State by Act No. 41 of 2013

	Product Name	Sale Price	Mrp	Number Of Ratings	Sale Date
0	APPLE iPhone 8 Plus (Gold, 64 GB)	49900.0	49900.0	3431	'14/1/2023'
1	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	1.0	3431	'15/1/2023'
2	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	84900.0	3431	'16/1/2023'
3	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	11202	'17/1/2023'
4	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	11202	'17/1/2023'
8	APPLE iPhone XS Max (Silver, 64 GB)	89900.0	89900.0	1454	'22/1/2023'

inplace = True

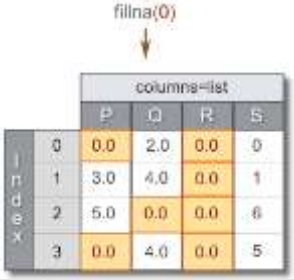
```
By default, the dropna() method returns a new DataFrame,
and will not change the original.
If you want to change the original DataFrame, use the
inplace = True argument
```

```
df.dropna(inplace=True)
```

Replace Empty Values

fillna()

```
replace empty cells with a new value
```




```
#row 5,6,7 null value change to 99

df.fillna(999)
```

PRESIDEN
UNIVERSI
Private University Estd. in Karnataka State by Act No. 41

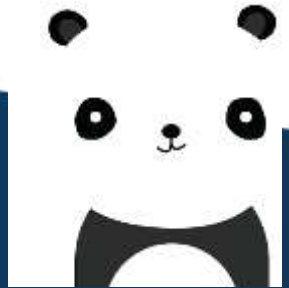
	Product Name	Sale Price	Mrp	Number Of Ratings	Sale Date
0	APPLE iPhone 8 Plus (Gold, 64 GB)	49900.0	49900.0	3431	'14/1/2023'
1	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	1.0	3431	'15/1/2023'
2	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	84900.0	3431	'16/1/2023'
3	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	11202	'17/1/2023'
4	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	11202	'17/1/2023'
5	APPLE iPhone 8 Plus (Silver, 64 GB)	999.0	49900.0	3431	'19/1/2023'
6	APPLE iPhone 8 Plus (Space Grey, 64 GB)	49900.0	999.0	3431	20/1/2023
7	APPLE iPhone 8 (Space Grey, 256 GB)	77000.0	77000.0	11202	999
8	APPLE iPhone XS Max (Silver, 64 GB)	89900.0	89900.0	1454	

Replace Only For Specified Columns

```
df["Mrp"].fillna(879)

# row 6 changed of MRP
```

```
0    49900.0
1        1.0
2    84900.0
3    77000.0
4    77000.0
5    49900.0
6      879.0
7    77000.0
8    89900.0
Name: Mrp, dtype: float64
```

PRESIDENCY UNIVERSITY
GAIN MORE KNOWLEDGE REACH GREATER HEIGHTS
Private University Estd. in Karnataka State by Act No. 41 of 2013



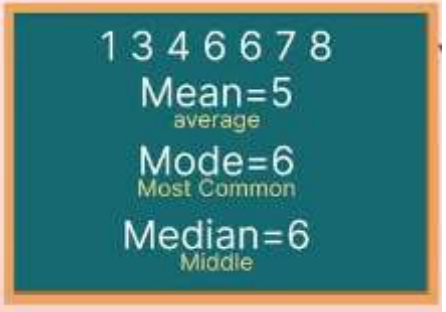
Replace Using Mean, Median, or Mode

calculate the respective values for a specified column

e.g. calculate MEAN, and replace any empty values with it

```
# Mean = the average value

x = df["Mrp"].mean()

df["Mrp"].fillna(x)
# MRP row 6 changed
```

```
0    49900.000
1        1.000
2    84900.000
3    77000.000
4    77000.000
5    49900.000
6    63200.125
7    77000.000
8    89900.000
Name: Mrp, dtype: float64
```



calculate the MEDIAN, and replace any empty values with it

```
# Median = the value in the middle

x= df["Sale Price"].median()
```

PRESIDEN
UNIVERSI
Private University Estd. in Karnataka State by Act No. 41

```python
df["Sale Price"].fillna(x)
# 'Sale Price' row 5 changed
```

```
0    49900.0
1    84900.0
2    84900.0
3    77000.0
4    77000.0
5    77000.0
6    49900.0
7    77000.0
8    89900.0
Name: Sale Price, dtype: float64
```

calculate the MODE, and replace any empty values with it

```python
# Mode = most frequent value

x = df["Mrp"].mode()[0]

df["Mrp"].fillna(x)
# MRP row 6 changed
```

```
0    49900.0
1        1.0
2    84900.0
3    77000.0
4    77000.0
5    49900.0
6    77000.0
7    77000.0
8    89900.0
Name: Mrp, dtype: float64
```

PRESIDENCY UNIVERSITY
GAIN MORE KNOWLEDGE REACH GREATER HEIGHTS
Private University Estd. in Karnataka State by Act No. 41

Cleaning Data of Wrong Format

```
# Non-date format column

df['Sale Date']
```

```
0    '14/1/2023'
1    '15/1/2023'
2    '16/1/2023'
3    '17/1/2023'
4    '17/1/2023'
5    '19/1/2023'
6      20/1/2023
7            NaN
8    '22/1/2023'
Name: Sale Date, dtype: object
```

Convert Into a Correct Format

to_datetime()

```
df['Sale Date'] =
pd.to_datetime(df['Sale Date'])
```

```
# column in date format
#NaT (Not a Time) i.e. empty cell

df['Sale Date']
```

PRESIDENCY UNIVERSITY
GAIN MORE KNOWLEDGE REACH GREATER HEIGHTS
Private University Estd. in Karnataka State by Act No. 41 of 2013
YEARS OF ACADEMIC WISDOM

```
0   2023-01-14
1   2023-01-15
2   2023-01-16
3   2023-01-17
4   2023-01-17
5   2023-01-19
6   2023-01-20
7          NaT
8   2023-01-22
Name: Sale Date, dtype: datetime64[ns]
```

Fixing Wrong Data

Two way - replace or remove

Replacing Values

```
# Mrp 2nd row incorrect value = 1

df['Mrp']
```

```
0    49900.0
1        1.0
2    84900.0
3    77000.0
4    77000.0
5    49900.0
6        NaN
7    77000.0
8    89900.0
Name: Mrp, dtype: float64
```

```
# change the value of 2nd row

df.loc[1, 'Mrp'] = 69999
```

PRESIDEN
UNIVERSI
Private University Estd. in Karnataka State by Act No. 41

```python
df['Mrp']
```

```
0    49900.0
1    69999.0
2    84900.0
3    77000.0
4    77000.0
5    49900.0
6        NaN
7    77000.0
8    89900.0
Name: Mrp, dtype: float64
```

Replace value by create some rules

```python
# Ensure MRP is at least 25k

for x in df.index:
    if df.loc[x,'Mrp'] < 25000:
        df.loc[x,'Mrp'] = 25000
print(df['Mrp'])
```

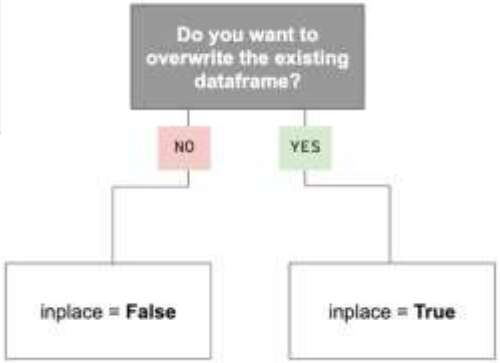
```
0    49900.0
1    25000.0
2    84900.0
3    77000.0
4    77000.0
5    49900.0
6        NaN
7    77000.0
8    89900.0
Name: Mrp, dtype: float64
```

Removing Rows of wrong data

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013

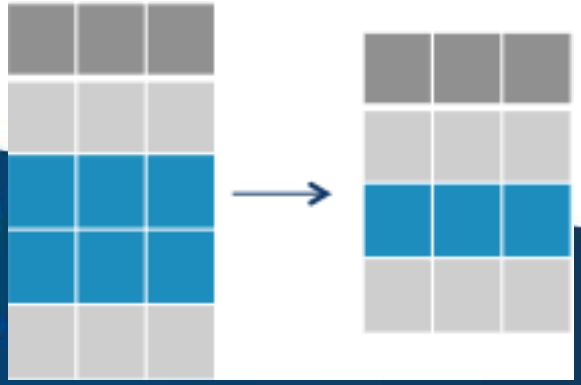
```python
for x in df.index:
    if df.loc[x,'Mrp'] < 25000:
        df.drop(x, inplace=True)

#'inplace=True':modify original DF

print(df['Mrp'])
```

```
0    49900.0
2    84900.0
3    77000.0
4    77000.0
5    49900.0
6        NaN
7    77000.0
8    89900.0
Name: Mrp, dtype: float64
```



Removing Duplicates

```python
df # row 3 & 4 duplicate
```



PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
40 YEARS

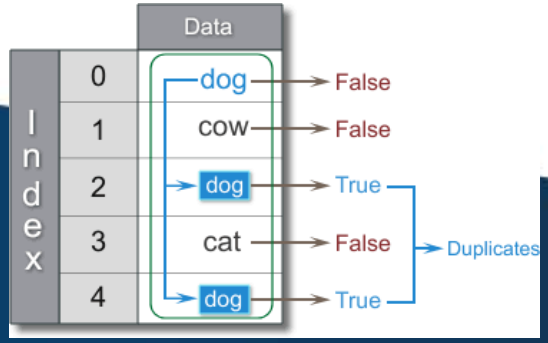
	Product Name	Sale Price	Mrp	Number Of Ratings	Sale Date
0	APPLE iPhone 8 Plus (Gold, 64 GB)	49900.0	49900.0	3431	'14/1/2023'
2	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	84900.0	3431	'16/1/2023'
3	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	11202	'17/1/2023'
4	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	11202	'17/1/2023'
5	APPLE iPhone 8 Plus (Silver, 64 GB)	NaN	49900.0	3431	'19/1/2023'
6	APPLE iPhone 8 Plus (Space Grey, 64 GB)	49900.0	NaN	3431	20/1/2023
7	APPLE iPhone 8 (Space Grey, 256 GB)	77000.0	77000.0	11202	NaN
8	APPLE iPhone XS Max (Silver, 64 GB)	89900.0	89900.0	1454	'22/1/2023'

duplicated()

to find duplicate row

```
# row 4 duplicate = True

print(df.duplicated())
```

```
0    False
2    False
3    False
4     True
5    False
6    False
7    False
8    False
dtype: bool
```



PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41

drop_duplicates()

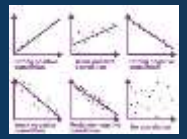
```
To remove duplicates
```

```python
# delete row 4

df.drop_duplicates()
```

	Product Name	Sale Price	Mrp	Number Of Ratings	Sale Date
0	APPLE iPhone 8 Plus (Gold, 64 GB)	49900.0	49900.0	3431	'14/1/2023'
2	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	84900.0	3431	'16/1/2023'
3	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	11202	'17/1/2023'
5	APPLE iPhone 8 Plus (Silver, 64 GB)	NaN	49900.0	3431	'19/1/2023'
6	APPLE iPhone 8 Plus (Space Grey, 64 GB)	49900.0	NaN	3431	20/1/2023
7	APPLE iPhone 8 (Space Grey, 256 GB)	77000.0	77000.0	11202	NaN
8	APPLE iPhone XS Max (Silver, 64 GB)	89900.0	89900.0	1454	'22/1/2023'

Data Correlations | corr()

```
Finding Relationships between column in your data set
The number varies from -1 to 1
1 means that there is a 1 to 1 relationship (a perfect
correlation)
```



PRESIDENCY UNIVERSITY
GAIN MORE KNOWLEDGE REACH GREATER HEIGHTS
YEARS OF ACADEMIC WISDOM
Private University Estd. in Karnataka State by Act No. 41 of 2013

```
df[['Sale Price',
    'Mrp',
    'Number Of Ratings']].corr()
```

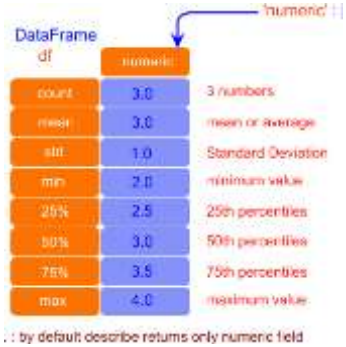
	Sale Price	Mrp	Number Of Ratings
Sale Price	1.000000	1.000000	0.177558
Mrp	1.000000	1.000000	0.177558
Number Of Ratings	0.177558	0.177558	1.000000

describe()

to view some basic statistical details like percentile, mean, std, etc. of a data frame or a series of numeric values

```
df.describe()
```

	Sale Price	Mrp	Number Of Ratings
count	7.000000	7.000000	8.000000
mean	72228.571429	72228.571429	6098.000000
std	16011.840857	16011.840857	4279.030932
min	49900.000000	49900.000000	1454.000000
25%	63450.000000	63450.000000	3431.000000
50%	77000.000000	77000.000000	3431.000000
75%	80950.000000	80950.000000	11202.000000
max	89900.000000	89900.000000	11202.000000



PRESIDENCY UNIVERSITY
GAIN MORE KNOWLEDGE REACH GREATER HEIGHTS
Private University Estd. in Karnataka State by Act No. 41 of 2013

Adding new Column in Dataframe

```
# DF[New Column Name]=Value

df['col'] = 5
```

```
df
```

	Product Name	Sale Price	Mrp	Number Of Ratings	Sale Date	col
0	APPLE iPhone 8 Plus (Gold, 64 GB)	49900.0	49900.0	3431	'14/1/2023'	5
1	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	1.0	3431	'15/1/2023'	5
2	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	84900.0	3431	'16/1/2023'	5
3	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	11202	'17/1/2023'	5
4	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	11202	'17/1/2023'	5
5	APPLE iPhone 8 Plus (Silver, 64 GB)	NaN	49900.0	3431	'19/1/2023'	5
6	APPLE iPhone 8 Plus (Space Grey, 64 GB)	49900.0	NaN	3431	20/1/2023	5
7	APPLE iPhone 8 (Space Grey, 256 GB)	77000.0	77000.0	11202	NaN	5
8	APPLE iPhone XS Max (Silver, 64 GB)	89900.0	89900.0	1454	'22/1/2023'	5

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
40 YEARS

Column Deletion in Dataframe

drop()

```
df.drop(['col','Number Of Ratings'
        axis=1,inplace = True)
```

```
df
```

	Product Name	Sale Price	Mrp	Sale Date
0	APPLE iPhone 8 Plus (Gold, 64 GB)	49900.0	49900.0	'14/1/2023'
1	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	1.0	'15/1/2023'
2	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	84900.0	'16/1/2023'
3	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	'17/1/2023'
4	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	'17/1/2023'
5	APPLE iPhone 8 Plus (Silver, 64 GB)	NaN	49900.0	'19/1/2023'
6	APPLE iPhone 8 Plus (Space Grey, 64 GB)	49900.0	NaN	20/1/2023
7	APPLE iPhone 8 (Space Grey, 256 GB)	77000.0	77000.0	NaN
8	APPLE iPhone XS Max (Silver, 64 GB)	89900.0	89900.0	'22/1/2023'

Row Addition

using concat()

```
a = pd.DataFrame({'Product Name':5
                  'Sale Price':6,
                  'Mrp':7,
                  'Sale Date':8},
```

PRESIDEN
UNIVERSI
Private University Estd. in Karnataka State by Act No. 41

```
                    index =[0])

a
```

	Product Name	Sale Price	Mrp	Sale Date
0	5	6	7	8

```
df = pd.concat([a, df])
```

```
df
```

	Product Name	Sale Price	Mrp	Sale Date
0	5	6.0	7.0	8
1	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	1.0	'15/1/2023'
2	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	84900.0	'16/1/2023'
3	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	'17/1/2023'
4	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	'17/1/2023'
5	APPLE iPhone 8 Plus (Silver, 64 GB)	NaN	49900.0	'19/1/2023'
6	APPLE iPhone 8 Plus (Space Grey, 64 GB)	49900.0	NaN	20/1/2023
7	APPLE iPhone 8 (Space Grey, 256 GB)	77000.0	77000.0	NaN
8	APPLE iPhone XS Max (Silver, 64 GB)	89900.0	89900.0	'22/1/2023'

Row Deletion in DataFrame

```
df.drop(0,inplace = True)
# index 0 deleted
```

PRESIDENCY UNIVERSI
GAIN MORE KNOWLEDGE REACH GREATER HEIGHTS
Private University Estd. in Karnataka State by Act No. 41

```
df
```

	Product Name	Sale Price	Mrp	Sale Date
1	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	1.0	'15/1/2023'
2	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	84900.0	'16/1/2023'
3	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	'17/1/2023'
4	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	'17/1/2023'
5	APPLE iPhone 8 Plus (Silver, 64 GB)	NaN	49900.0	'19/1/2023'
6	APPLE iPhone 8 Plus (Space Grey, 64 GB)	49900.0	NaN	20/1/2023
7	APPLE iPhone 8 (Space Grey, 256 GB)	77000.0	77000.0	NaN
8	APPLE iPhone XS Max (Silver, 64 GB)	89900.0	89900.0	'22/1/2023'

Slicing in Pandas

```
df[2:5]    # Start : STOP
```

	Product Name	Sale Price	Mrp	Sale Date
3	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	'17/1/2023'
4	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	'17/1/2023'
5	APPLE iPhone 8 Plus (Silver, 64 GB)	NaN	49900.0	'19/1/2023'

```
df[1:7:2]    # Start : STOP : Step
```

	Product Name	Sale Price	Mrp	Sale Date
2	APPLE iPhone 8 Plus (Space Grey, 256 GB)	84900.0	84900.0	'16/1/2023'
4	APPLE iPhone 8 (Silver, 256 GB)	77000.0	77000.0	'17/1/2023'
6	APPLE iPhone 8 Plus (Space Grey, 64 GB)	49900.0	NaN	20/1/2023

```
df[['Mrp']][4:11]    # Start : STOP
```

	Mrp
5	49900.0
6	NaN
7	77000.0
8	89900.0

What is Matplotlib?

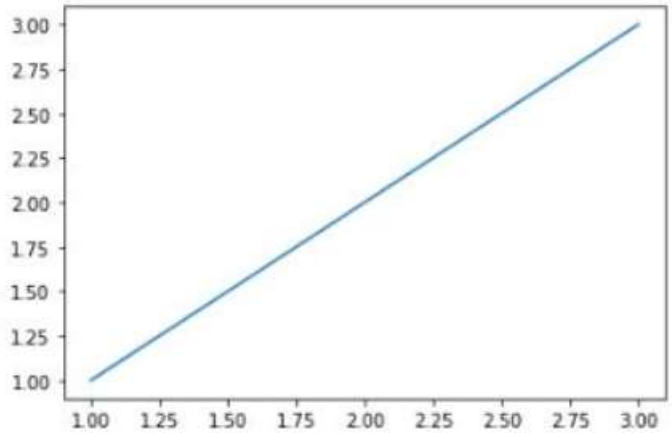
- Matplotlib is an open-source drawing library that supports various drawing types

- You can generate plots, histograms, bar charts, and other types of charts with just a few lines of code

- It's often used in web application servers, shells, and Python scripts

Pyplot is a Matplotlib module that provides simple functions for adding plot elements, such as lines, images, text, etc. to the axes in the current figure.

```
In [3]: from matplotlib import pyplot as plt

        #plotting to our canvas
        plt.plot([1,2,3],[1,2,3])

        #showing what we plotted
        plt.show()
```



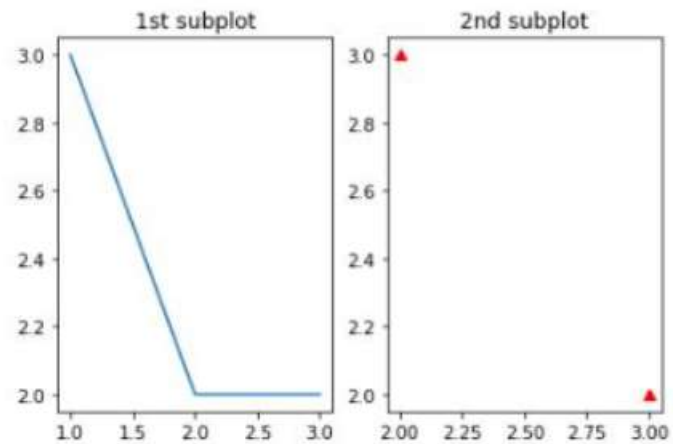
Matplotlib Subplots

You can use the subplot() method to add more than one plot in a figure.

Syntax: plt.subplots(nrows, ncols, index)

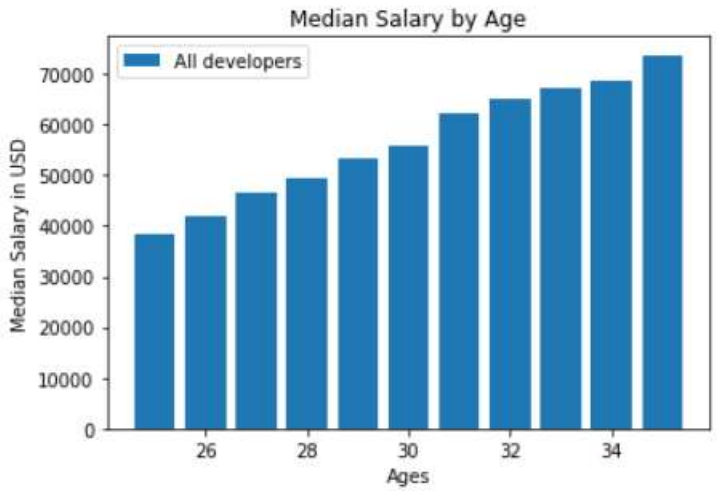
The three-integer arguments specify the number of rows and columns and the index of the subplot grid.

```
In [1]: from matplotlib import pyplot as plt
        plt.subplot(1,2,1)
        plt.plot([1,2,3],[3,2,2])
        plt.title("1st subplot")

        plt.subplot(1,2,2)
        plt.plot([2,3,3],[3,2,2],"r^")
        plt.title("2nd subplot")
        plt.show()
```



1. Plotting the bar plot

```
[ ] plt.bar(x, devs_y, label="All developers")
    plt.xlabel("Ages")
    plt.ylabel("Median Salary in USD")
    plt.title("Median Salary by Age")
    plt.legend()
    plt.show()
```



1. Plotting the Pie Chart

```
[ ] slices = [30, 40, 20, 50] #sum needs not be 100
```

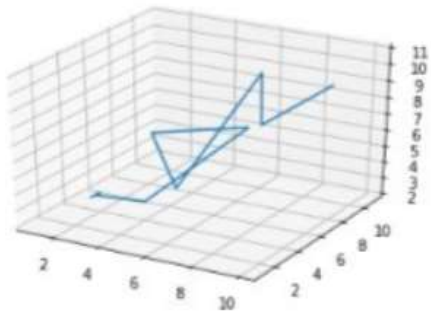
```
[ ] plt.pie(slices)
    plt.show()
```

Matplotlib Three-Dimensional Plotting

Importing the mplot3d toolkit enables users to create three-dimensional plots.

Example:

```
In [4]: from mpl_toolkits.mplot3d import axes3d
        import matplotlib.pyplot as plt
        import numpy as np

        fig = plt.figure()
        ax = fig.add_subplot(projection='3d')
        X, Y, Z = [1,2,3,4,5,6,7,8,9,10],[5,4,2,4,11,1,1,6,4,8],[2,3,4,3,5,9,6,11,9,10]
        ax.plot(X, Y, Z)
        plt.show()
```



Thank you!